Ders 1 Çalışma Özeti

Ders 1 Çalışma Özeti

Tarih: 19 Mart 2026 Perşembe. Ramazan Bayramı arifesinde online yapılan ders. Süre yaklaşık 1 saat 40 dakika (arada ara var). Slaytlar Prof. Dr. M. Fatih Amasyalı'nın Yapay Zeka ders notlarından.

Genel Konular

- Kör (bilgisiz, uninformed) arama algoritmalarının kısa tekrarı: BFS, DFS, düşük maliyetli arama (uniform cost search, UCS).
- Kör aramanın sınırları: Rastgele oluşturulmuş bir yol haritasında algoritmaların adım sayısı ve yol maliyeti karşılaştırması (Python uygulaması).
- Sezgisel arama (heuristic search) fikri ve sezgisel fonksiyon $h(n)$.
- En iyi ilk arama (best-first search, açgözlü arama / greedy search): $f(n) = h(n)$.
- A* araması: $f(n) = g(n) + h(n)$.
- Romanya haritası örneği: Arad'dan Bükreş'e (Bucharest) gitmek.
- Izgara (grid) üzerinde Manhattan uzaklığıyla best-first ve A* karşılaştırması.
- 8 taş (8-puzzle) problemi için iki sezgisel fonksiyon: h_1 ve h_2.
- S'den G'ye giden şehir grafı üzerinde BFS, DFS, UCS ve A* izlemesi. Sınavda beklenen tablo biçimi.
- A*'ın optimalliği: kabul edilebilir (admissible) sezgisel fonksiyon.
- Uygulama: çok robotlu keşifte A* kullanımı (makale videosu).

Hocanın Özellikle Vurguladığı Kısımlar

- Kör algoritmalar hedefe çok yaklaşıp sonra uzaklaşabilir. Çünkü hiçbir bilgiye bakmadan, yalnız verilen direktife göre düğüm seçerler.
- BFS yalnız her adımın maliyeti eşitse optimal çözümü bulur. Maliyetler farklıysa bulamayabilir.
- UCS optimal çözümü bulur ama çok fazla düğüm açar. Arama uzayı büyüdükçe yönetilemez hâle gelir. Yapısı Dijkstra algoritmasına benzer.
- Sezgisel bilgi "hedefe ne kadar kaldı?" sorusunun tahminidir. Tahmin iyimser olmalı, yani gerçek maliyeti aşmamalı.
- Best-first search hızlı bir çözüm bulur ama yerel minimuma takılabilir. Bulduğu çözüm optimal olmayabilir.
- Best-first search tam (complete) değildir. Hedefe eşit uzaklıktaki iki durum arasında döngüye girebilir.
- A*, harcanan maliyeti (g) ve kalan tahmini maliyeti (h) birlikte kullanır. Böylece yerel minimumlardan kurtulur.
- Her A* optimal değildir. Optimallik için $h(n)$ iyimser seçilmeli ve tüm maliyetler pozitif olmalı.
- Sınavda A* sorusunda her adımda kuyruğun (açık listenin) durumu ve kuyruktan çekilen eleman yazılmalı. Gelişigüzel bir çözüm kabul edilmez.
- Harita problemlerinde iyi bir sezgisel fonksiyon kuş uçuşu (straight-line, Öklid) uzaklıktır.

Kısa Tekrar Notları

- Arama stratejisi değerlendirme ölçütleri: tamlık (completeness), zaman karmaşıklığı, bellek (space) karmaşıklığı, optimallik.

- b: en fazla dallanma sayısı (branching factor). d: en düşük maliyetli çözümün derinliği. m: arama uzayının en büyük derinliği (sonsuz olabilir).
- BFS kuyruk (FIFO) kullanır. Yeni durumlar kuyruğun sonuna eklenir. Başa eklenirse DFS olur.
- UCS öncelikli kuyruk (priority queue) kullanır. Ölçüt: başlangıçtan bu düğüme kadar harcanan maliyet.
- Best-first search: $f(n) = h(n)$. Kuyruktaki durumlar h değerine göre sıralanır.
  - Tam değil, optimal değil.
  - Zaman ve bellek: $O(b^m)$. İyi bir sezgisel fonksiyon zamanı çok azaltır.
- A*: $f(n) = g(n) + h(n)$.
  - $g(n)$: başlangıçtan n'e kadar gerçek maliyet.
  - $h(n)$: n'den hedefe tahmini maliyet.
  - Tam ve (kabul edilebilir h ile) optimal.
- Romanya örneği:
  - Best-first: Arad → Sibiu → Fagaras → Bükreş, maliyet $140 + 99 + 211 = 450$.
  - A*: Arad → Sibiu → Rimnicu Vilcea → Pitesti → Bükreş, maliyet $140 + 80 + 97 + 101 = 418$ (optimal).
- 8 taş problemi:
  - h_1: yerinde olmayan taş sayısı.
  - h_2: taşların hedef konumlarına Manhattan uzaklıkları toplamı.
  - Her durumda $h_2(n) \geq h_1(n)$ ise h_2 baskındır (dominant) ve kullanılması daha iyidir.

Detaylı Açıklamalar

Kör aramanın tekrarı ve karşılaştırma uygulaması

- Hoca, MATLAB'da yazılmış bir karşılaştırma kodunu Python'a çevirmiş. Kod rastgele bir graf (ör. 20 düğüm, 35 yol) oluşturuyor ve BFS, DFS, UCS, best-first ve A*'ı çalıştırıyor.
- Slayttaki örnekte 12 numaralı düğümden 1'e gidiliyor:
  - BFS 37 adımda 128 maliyetli bir yol buluyor. Yol 5 atlamalı ama en ucuz değil.
  - UCS 110 maliyetli optimal yolu buluyor ama çok daha fazla adım atıyor.
  - Best-first ve A* aynı optimal sonuca çok daha az adımda ulaşıyor.
  - DFS hedefin "kapısına kadar" gelip uzaklaşıyor. Bulduğu yol optimal değil.
- Canlı denemelerde de A* ve best-first, hem adım sayısı hem yol maliyeti açısından öne çıkıyor.
- Statik ve küçük graflarda kör algoritmalar iş görebilir. Dinamik ve sürekli genişleyen gerçek dünya problemlerinde daha hızlı yöntemler gerekir.

Sezgisel arama

- Durum uzayı, operatörler, başlangıç ve hedef durumu kör aramadakiyle aynıdır. Fark: bir durumdan diğerine geçiş sezgisel bir kuralla seçilir.
- Feldman ve Feigenbaum'un tanımına göre sezgisellik, durum uzayı çok büyük veya dinamik olduğunda aramayı sınırlayan kural, strateji ve sadeleştirmelerdir.
- Özet: Bir sonraki düğümü, beni hedefe daha hızlı götürecek tahmine göre seçerim.

En iyi ilk arama (best-first search)

- Algoritma:
  1. Kök düğümü kuyruğa koy.
  2. Kuyruk boş değilken ilk durumu çek. Hedefse dur.
  3. Çekilen durumdan gidilebilecek tüm durumları kuyruğa ekle.
  4. Kuyruktaki durumları değerlendirme fonksiyonuna göre puanla ve küçükten büyüğe sırala.
- UCS'deki maliyet, şimdiye kadar harcanan efordur. Best-first'teki $h(n)$ ise hedefe kalan tahmini uzaklıktır.
- Romanya örneği (kuş uçuşu uzaklıklar: Arad 366, Sibiu 253, Timisoara 329, Zerind 374, Fagaras 176, Rimnicu Vilcea 193):
  - Arad açılır. Sibiu (253) en küçük olduğu için seçilir.
  - Sibiu açılır. Fagaras (176) seçilir.

  - Fagaras açılır. Bükreş ($h = 0$) seçilir ve arama biter.
  - Toplam 4 adımda çözüm bulunur, maliyet 450. Optimal yol 418 olduğu için algoritma yerel minimuma takılmıştır.
- Izgara örneği: Engeller dikkate alınmadan her karenin hedefe Manhattan uzaklığı yazılır. Best-first, h değeri küçük görünen karelere yönelir. Engelin arkasında dolanarak uzun bir yol bulur. Yol bulunur ama optimal değildir.

A* araması

- A*, UCS ile best-first aramanın birleşimi gibi düşünülebilir.
- Romanya örneği (Python ile doğrulandı):

Açılan düğüm	$f = g + h$
Arad	$0 + 366 = 366$
Sibiu	$140 + 253 = 393$
Rimnicu Vilcea	$220 + 193 = 413$
Fagaras	$239 + 176 = 415$
Pitesti	$317 + 100 = 417$
Bükreş	$418 + 0 = 418$

- Fagaras üzerinden Bükreş'e $f = 450$ ile ulaşılır. Ama kuyrukta 418'lik bir seçenek olduğu için A* onu seçer.
- Döngüler kendiliğinden elenir. Örneğin Arad → Sibiu → Arad yolunda $f = 280+366 = 646$ olur. Bu değer diğer seçeneklerden büyük olduğu için seçilmez.
- Izgara örneğinde A* her karede $g+h$ değerini hesaplar. Bir yolda f büyüyünce kuyruktaki daha ucuz seçeneğe döner. Sonunda optimal yolu bulur. Açılan kare sayısı yine de az değildir, ama BFS veya UCS'den çok daha azdır.

8 taş problemi için sezgisel fonksiyonlar

- Başlangıç durumu (slayt): 7 2 4 / 5 _ 6 / 8 3 1.
  - $h_1 = 8$: 8 taşın hiçbiri yerinde değil.
  - $h_2 = 18$: taşların hedefe Manhattan uzaklıkları toplamı.
- Her hamle (boşluğu yukarı, aşağı, sağa, sola kaydırma) 1 birim maliyetlidir (g).
- Deneysel sonuçlar (açılan düğüm sayısı, kitaptaki tablo):
  - Derinlik 12: IDS yaklaşık 3,6 milyon; A*(h_1) 227; A*(h_2) 73.
  - Derinlik 24: IDS hesaplanamayacak kadar çok; A*(h_1) 39.135; A*(h_2) 1.641.
- Esnetilmiş problem (relaxed problem) örneği: siyah (B) ve beyaz (W) taşların yer değiştirmesi. Bir taş yanındaki boşluğa kayabilir veya bir taşın üzerinden atlayabilir. Bu problem için bir $h(n)$ önermek çalışma sorusu olarak verildi.

Şehir grafı örneği (S'den G'ye)

- Şekil-1'deki kenar maliyetleri: S–A 3, S–D 4, A–B 4, A–D 5, B–C 4, B–E 5, D–E 2, E–F 4, F–G 3.
- G'ye kuş uçuşu uzaklıklar: S 11, A 10,4, B 6,7, C 4, D 8,9, E 6,9, F 3, G 0.
- BFS ve DFS farklı derinliklerde farklı G düğümleri bulur. Maliyetler eşit olmadığı için ilk bulunan çözüm optimal olmayabilir. DFS bellek açısından doğrusal ve tasarrufludur.
- UCS optimal yolu bulur ama bellek kullanımı yüksektir.
- A* izlemesi (Python ile doğrulandı): S (11) → D ($4 + 8{,}9 = 12{,}9$) → E ($6 + 6{,}9 = 12{,}9$) → F ($10 + 3 = 13$) → G (13). Yol S–D–E–F–G, maliyet 13.
- Sınav için tablo biçimi: Her satırda kuyruktaki tüm yollar ve $h + g$ değerleri yazılır. Kuyruktan çekilen yol kalın yazılır veya işaretlenir.

A*'ın optimalliği

- Koşul: $h(n)$ gerçek kalan maliyeti aşmamalı (iyimser, kabul edilebilir) ve tüm adım maliyetleri pozitif olmalı.

- G optimal hedef, G_2 alt-optimal (suboptimal) bir hedef olsun. n, kuyrukta G'ye giden optimal yol üzerindeki bir düğüm olsun.
  - $h(G_2) = 0$ olduğu için $f(G_2) = g(G_2) > g(G)$.
  - h kabul edilebilir olduğu için $f(n) = g(n) + h(n) \leq g(G)$.
  - Böylece $f(n) < f(G_2)$ olur. A*, G_2'yi n'den önce açmaz ve optimal hedefe ulaşır.

Uygulama: robot keşfi

- Bir ortamda hedefe gitmeden önce ortamın haritası çıkarılmalıdır. Örneğin yangını söndürecek bir robot önce haritayı bilmelidir.
- Gösterilen 2019 tarihli çalışmada çok robotlu keşifte A* kullanılmış. Makalede kullanılan h ve f fonksiyonları ayrıntılı veriliyor.
- A'*ın değiştirilmiş birçok türü vardır. Hocanın danışmanlık yaptığı bir öğrenci ekibi, dronlar üzerinde A*'ın bir türevini kullanarak Teknofest'te en iyi algoritma ödülü almış.

Ders sonrası notlar

- Altyazıda "BFS" kısaltması bazen best-first search için de kullanılıyor. Hoca bunu fark edip best-first search için "BeFS" denmesini önerdi. Bu özette BFS yalnız genişlik öncelikli arama (breadth-first search) anlamındadır.
- Hoca, h_2'nin baskın olduğunda "daha iyimser" bilgi verdiğini söylüyor. Doğrusu: Baskın sezgisel fonksiyon gerçek maliyete daha yakındır, yani daha az iyimserdir. İkisi de kabul edilebilir olduğu sürece daha yakın tahmin daha az düğüm açtırır.
- Sınav tablosu slaytındaki (A* Arama) graf, Şekil-1'den farklı kenar maliyetleri kullanıyor: B–E 1, D–E 5. Bu grafta optimal yol S–A–B–E–F–G, maliyet $3+4+1+4+3 = 15$. Hocanın derste söylediği "15 birim maliyetle S, A, B, E, F, G" sonucu bu slayta aittir. Şekil-1'deki maliyetlerle aynı yolun maliyeti 19, optimal yol ise S–D–E–F–G (13) olur.
- A* tamlık ve optimallik özellikleri, ağaç aramasında kabul edilebilir h ile geçerlidir. Kapalı liste (tekrar edilen durumları atlama) kullanılan graf aramasında ayrıca tutarlılık (consistency) koşulu gerekir. Bu ayrıntı derste geçmedi; kitapta (Russell ve Norvig) anlatılıyor.
- Romanya tablosunda Pitesti'nin kuş uçuşu uzaklığı 100'dür. Slayt karesinde sayı kısmen görünüyor.

Duyurular

- Gelecek hafta yerel arama (local search) algoritmaları işlenecek: doğadan esinlenen yöntemler, genetik algoritma, fonksiyonların minimum/maksimum noktalarını bulma.
- Konuyu kitaptan (3. bölüm) tekrar edin. Labirentte BFS, DFS, UCS, best-first ve A*'ın hangi kareleri deneyeceği çalışma sorusu olarak verildi.

Not: İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.